

第 27 讲实验设计文档

PyTorch Tensor: NumPy 对照、dtype、device 与 autograd 预览

【实验目标】 不是“熟悉 PyTorch API 列表”，而是验证一个更重要的判断：从 NumPy 进入 PyTorch 后，Shape 与矩阵运算直觉基本连续；PyTorch 新增的关键维度是 dtype、device 和 autograd 元数据。

1 实验问题

本实验围绕 6 个问题：

1. 同一组数字放进 NumPy 和 PyTorch，Shape 和数值是否一致？
2. PyTorch 的索引、切片、矩阵乘法是否延续 NumPy 规则？
3. NumPy 与 PyTorch 默认 dtype 是否可能不同？
4. from_numpy 与 torch.tensor 是否具有不同的内存行为？
5. device 与 requires_grad 能否作为 Tensor 元数据直接观察？
6. LLM 风格 (B, T, D) Tensor 是否继续服从 $(B, T, D) @ (D, O) \rightarrow (B, T, O)$ ？

2 实验假设

编号	假设	预计验证方式
H1	NumPy 与 PyTorch 可表示相同数值和 Shape	创建相同矩阵并比较
H2	索引与矩阵 Shape 规则高度一致	切片、@ 直接观察
H3	默认浮点 dtype 可能不同	打印 dtype
H4	from_numpy 常共享 CPU 内存, torch.tensor 常复制	修改一侧数据观察另一侧
H5	打开梯度跟踪后，后续结果会记录计算依赖；未 backward 时 grad 仍为空	查看 requires_grad、grad_fn、grad
H6	三维 LLM Tensor 的 Linear-like 运算仍沿最后特征维进行	$(2, 3, 4) @ (4, 5)$ 验证

3 环境与运行方式

本材料包已在以下环境实际运行：

- Python 3.13.5
- NumPy 2.3.5
- PyTorch 2.10.0+cpu
- 当前验证环境 CUDA 不可用，因此 device 实验实际走 CPU 路径；代码已写成有 CUDA 时自动选择 CUDA。

运行：

```
python lesson27_pytorch_tensor_experiment.py
```

4 实验 1：ndarray 与 Tensor 的基本对应

输入：

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

分别创建：

```
np_x = np.array(..., dtype=np.float32)
torch_x = torch.tensor(..., dtype=torch.float32)
```

观测：

- Shape 是否都是 (2, 3)；
- dtype 是否都为 32 位浮点；
- 数值是否逐元素相等。

实际结果：values equal: True。

5 实验 2：索引与 Shape

对 $X.shape = (2, 3)$ 依次执行：

```
X[0]
X[0:1]
X[:,1]
X[:,1:2]
```

预计：

(3,), (1, 3), (2,), (2, 1)

实际结果完全一致。

这一步的意义是确认：第 2 讲建立的“整数索引可能删维、切片保留维度”直觉进入 PyTorch 后仍然有效。

6 实验 3：逐元素运算与矩阵乘法

构造：

$$X.shape = (2, 3), \quad B.shape = (2, 3), \quad W.shape = (3, 2)$$

比较：

```
X * B
X @ W
```

结果：

$$X * B : (2, 3)$$

$$X @ W : (2, 2)$$

7 实验 4：dtype 对照

分别执行：

```
np.array([1.0, 2.0, 3.0])
torch.tensor([1.0, 2.0, 3.0])
```

实际环境结果：

- NumPy: float64
- PyTorch: torch.float32

这里的结论不是“NumPy 永远 float64、PyTorch 永远 float32”，而是：不要依赖默认值。科研实验需要显式检查和必要时显式指定 dtype。

8 实验 5：内存共享与复制

准备：

```
a = np.array([1., 2., 3.], dtype=np.float32)
t_shared = torch.from_numpy(a)
t_copy = torch.tensor(a)
```

步骤：

1. 先修改 `a[0]=99`;
2. 再修改 `t_shared[1]=77`;
3. 同时观察三者。

实际：

修改 `a[0]` 后:

```
numpy = [99. 2. 3.]
shared = tensor([99., 2., 3.])
copy = tensor([1., 2., 3.])
```

修改 `shared[1]` 后:

```
numpy = [99. 77. 3.]
shared = tensor([99., 77., 3.])
copy = tensor([1., 2., 3.])
```

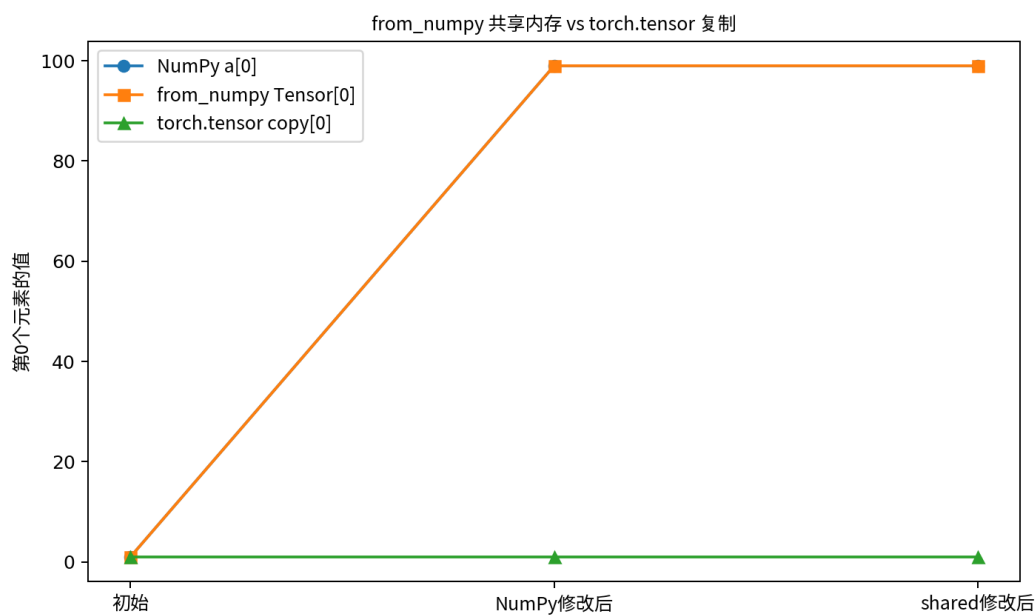


图 1: 共享内存与复制的实际观测

9 实验 6: device

代码自动选择:

```
device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)
x_device = torch_x.to(device)
```

当前验证环境: `device=cpu`。

如果你在自己的 RTX 4060 机器上运行, 并且 PyTorch CUDA 环境配置正确, CUDA 检测可能返回 `True`, 代码就会自动选择 GPU。实验结果中的 `device` 与当前机器环境有关, 不属于固定理论常数。

10 实验 7: requires_grad 与计算图预览

构造:

```
x = torch.tensor([2.0, -1.0], requires_grad=True)
y = x * 3.0 + 1.0
z = y ** 2
```

观察：

- `x.requires_grad=True`
- `y.requires_grad=True`
- `z.requires_grad=True`
- `x.is_leaf=True`
- `y.is_leaf=False`
- `z.grad_fn=PowBackward0`
- 尚未 backward，因此 `x.grad=None`

再用：

```
with torch.no_grad():
    y2 = x * 3.0 + 1.0
```

观察 `y2.requires_grad=False`。

11 实验 8：LLM 风格 Tensor

构造：

$$X.shape = (2, 3, 4)$$

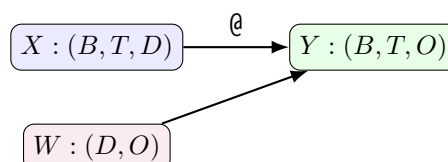
$$W.shape = (4, 5)$$

执行：

$$Y = X @ W$$

实际得到：

$$Y.shape = (2, 3, 5)$$



12 实验结果如何解释？

本实验支持以下结论：

1. PyTorch 没有推翻 NumPy 的基础 Tensor/Shape 心智模型；
2. PyTorch 的新增核心是“深度学习执行环境元数据”：dtype、device、autograd；
3. 同一个 Tensor 的数值内容之外，还必须检查它在哪里、什么精度、是否追踪梯度；
4. LLM 的三维 hidden states 在 PyTorch 中仍服从前面学过的矩阵规则。

13 实验局限

- 当前环境是 CPU-only，未实测 CUDA 速度与显存；
- 本讲只预览 autograd，没有正式执行 backward()；
- 没有比较 FP16/BF16 的硬件性能差异；
- 没有进入 nn.Module、Optimizer、DataLoader。

这些内容会在后续课程按需补齐。

14 实验验收

完成后，你应该能够独立回答：

1. 为什么 Tensor 不只是“数组换了名字”？
2. 为什么 dtype/device mismatch 是实际实验中的常见故障源？
3. 为什么 from_numpy 需要警惕共享内存副作用？
4. 为什么 requires_grad=True 与 backward() 必须分开理解？
5. 为什么 (B, T, D) 进入 PyTorch 后仍然能沿用前 6 讲的 Shape 推理？